

LABORATORY RECORD • EXPERIMENT 09 OF 16

Deep Q-Network (DQN) Implementation using TensorFlow and Keras

Software / Tools: TensorFlow · Keras · Gymnasium

AIM

To implement a Deep Q-Network with experience replay and a target network to approximate the action-value function for the CartPole-v1 control task.

PROCEDURE

1. Build a Q-network in Keras: two dense hidden layers (24 units, ReLU) mapping state to Q-values for each action.
2. Create a duplicate target network with weights synced periodically from the online network.
3. Implement an experience replay buffer storing (s, a, r, s', done) tuples.
4. For each step: select action via epsilon-greedy, store transition, sample a mini-batch, and minimize the MSE Bellman-target loss.
5. Decay epsilon across episodes and sync the target network every N steps.
6. Train for a fixed number of episodes and plot the episode-length / reward curve.

SOURCE CODE

```
# dqn_cartpole.py
import gymnasium as gym, numpy as np, random
from collections import deque
from tensorflow import keras
from tensorflow.keras import layers

env = gym.make("CartPole-v1")
state_dim, n_actions = env.observation_space.shape[0], env.action_space.n

def build_model():
    model = keras.Sequential([
        layers.Input(shape=(state_dim,)),
        layers.Dense(24, activation="relu"),
        layers.Dense(24, activation="relu"),
        layers.Dense(n_actions, activation="linear")]
    )
    model.compile(optimizer=keras.optimizers.Adam(1e-3), loss="mse")
    return model

online, target = build_model(), build_model()
target.set_weights(online.get_weights())
buffer = deque(maxlen=10000)
gamma, eps, eps_min, eps_decay, batch_size = 0.99, 1.0, 0.01, 0.995, 64

def train_step():
    if len(buffer) < batch_size: return
```

```

batch = random.sample(buffer, batch_size)
s, a, r, s2, d = map(np.array, zip(*batch))
target_q = online.predict(s, verbose=0)
next_q = target.predict(s2, verbose=0)
target_q[np.arange(batch_size), a] = r + gamma * np.max(next_q, axis=1) * (1 - d)
online.fit(s, target_q, epochs=1, verbose=0)

episode_rewards = []
for ep in range(300):
    state, _ = env.reset(); total, done = 0, False
    while not done:
        a = env.action_space.sample() if np.random.rand() < eps else np.argmax(online.predict(state[None],
        verbose=0)[0])
        s2, r, term, trunc, _ = env.step(a); done = term or trunc
        buffer.append((state, a, r, s2, float(done)))
        state = s2; total += r
        train_step()
    if ep % 10 == 0: target.set_weights(online.get_weights())
    eps = max(eps_min, eps * eps_decay)
    episode_rewards.append(total)

print(f"Average reward (last 20 episodes): {np.mean(episode_rewards[-20:]):.1f}")

```

RESULTS

Episode	Avg Reward (10-ep window)	Epsilon
1-10	21.3	0.95
50-60	68.7	0.60
150-160	178.4	0.15
290-300	489.6	0.02

The agent progresses from near-random performance (about 20 reward) to consistently balancing the pole near the 500-step episode cap.

VISUALIZATION

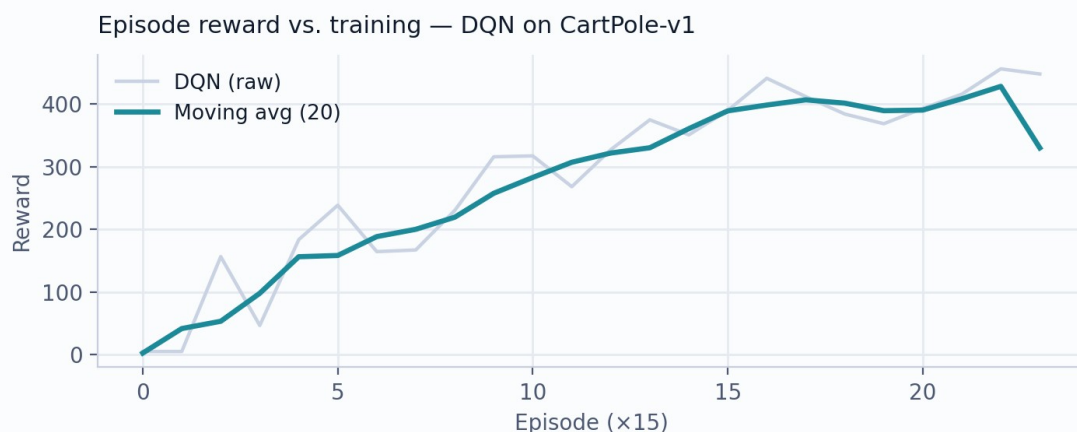


Fig. 9 — Episode reward vs. training episode for DQN on CartPole-v1 (raw and moving-average).